

ReliefLink

Next-Gen Geospatial Disaster Management & Humanitarian Relief Ecosystem

Project Scope: Full-Stack Academic Defense & Viva
Architecture

Architecture: MERN + Leaflet GIS +
WebSockets

Date:
2026

1. Project Introduction & Motivation

ReliefLink is an automated, real-time humanitarian coordination platform engineered to eliminate communication breakdown and logistical failures during climate crises and disaster emergencies (such as floods, cyclones, landslides, and earthquakes).

1.1 Main Motive & Core Objectives

- **Immediate Rescue Triage:** Provide affected individuals with a zero-friction, passwordless SOS distress lifeline backed by precise GPS coordinates.
- **Intelligent Supply Allocation:** Eliminate blind donations by dynamically routing donor aid to the highest-urgency, overcrowded relief camps using geographical distance and capacity thresholds.
- **Unified Emergency Synergy:** Connect four critical stakeholders (Affected Citizens, Relief Camps/NGOs, Field Volunteers, and Government Relief Authorities) into a single synchronized operational map.
- **Real-time Situational Transparency:** Broadcast geo-targeted weather warnings, evacuation camps, and safety radars directly from authoritative meteorological feeds (IMD).

1.2 Why It Differs (Novelty & Competitive Edge)

Unlike generic disaster portals or unstructured social media SOS posts (e.g., Twitter hashtags or WhatsApp broadcasts), ReliefLink introduces:

1. **Intelligent Haversine Camp Routing:** Rather than dumping goods at popular centers, donor items are scored using a weighted algorithm: $\text{Urgency} = (\text{Occupancy} / \text{Capacity}) \times 100 - (\text{Distance} \times 0.15)$.
2. **Interactive Satellite GIS Radar:** High-resolution Esri Satellite imagery displays true street layouts, relief camp markers, and live radius indicators, replacing confusing 3D abstractions with battle-tested mapping.
3. **Zero-Password Cellular Access:** Evacuees in shock cannot remember complex credentials; one-tap OTP authentication ensures immediate SOS dispatch within 5 seconds.
4. **Real-Time WebSocket Distress Broadcasts:** SOS beacons and emergency notices instantly trigger alert sirens across connected responder dashboards without manual page reloads.

2. Existing System vs. Proposed System

Criteria	Existing System (Manual / Fragmented)	Proposed System (ReliefLink)
SOS Reporting	Unverified phone calls to clogged helplines; unorganized social media tweets without coordinates.	One-tap OTP login with automated high-accuracy GPS capture and instant distress beaconing.
Camp Discovery	Paper announcements, word-of-mouth, or visiting multiple camps only to find them completely full.	Live Satellite Map displaying active camps, real-time occupancy percentages, and driving distances.
Donation Matching	Bottlenecks: one camp receives 10,000 water bottles while another camp lacks basic medicine.	Smart algorithmic routing that assigns donations to the highest-urgency camp within proximity.
Volunteer Dispatch	Uncoordinated crowds arrive on-site; volunteers lack verified mission locations and task statuses.	Role-based task management dashboard with skill verification, live status toggles, and route maps.

3. Key Problems Solved

- **Helpline Congestion:** Automates distress intake into structured, priority-ranked database records.
- **Geographic Ambiguity:** Replaces descriptive landmarks ("near the big banyan tree") with exact [latitude, longitude] coordinates.
- **Aid Wastage & Imbalance:** Directs public donations to verified NGO camps suffering inventory deficits.
- **Identity & Safety in Evacuation:** "Global Safety Radar" allows citizens to drop safe location pins so separated family members can verify their wellbeing.

4. Technology Stack & External Services

Layer	Technologies / Libraries	Role & Responsibility
Frontend Client	React 18, Vite, React Router DOM, Zustand, Lucide React	High-speed single page application (SPA), responsive UI, role-based dashboards, audio sirens.
Geospatial & Mapping	Leaflet, React-Leaflet, Esri World Imagery Tiles	Satellite terrain rendering, interactive pin dropping, cluster radius visualization, camp markers.
Backend API	Node.js, Express.js	RESTful routing, JWT authentication, validation middleware, HTTP proxying for government bulletins.
Real-Time Engine	Socket.io (WebSockets)	Bidirectional event streaming for live SOS alerts, broadcasts, and automated dashboard refreshes.
Database & Storage	MongoDB Atlas, Mongoose ODM	Document store featuring 2dsphere indexing for geospatial spherical distance queries.
Security & Auth	JSON Web Tokens (JWT), Bcrypt.js	Stateless authorization headers, salted one-way password hashing for responders, temporary OTP store.
External Services	HTML5 Geolocation API, IMD Weather Proxy, Mock Gateway	Real-time device GPS, official Indian Meteorological Dept bulletins, mock UPI payment confirmation.

5. Role-Wise Login & Signup Architecture

ReliefLink enforces a secure dual-authentication architecture: **Passwordless OTP** for affected victims in high-stress scenarios, and **Credentialed JWT** for operational responders.

Role 1: Citizen / Affected Evacuee (Phone + OTP Flow)

- 1 **Input:** User inputs mobile number (e.g. +91 9876543210) on the login card.
- 2 **Frontend Check:** Validates presence of digits and country dial-code formatting.
- 3 **API Dispatch:** Calls `POST /api/auth/send-otp` with `{ phone }`.
- 4 **Backend Verification & Generation:**
 - Database query checks if the phone exists: `User.findOne({ phone })`.
 - Generates a cryptographically random 6-digit OTP (e.g., `Math.floor(100000 + Math.random() * 900000)`).
 - Calculates expiration: `otpExpires = Date.now() + 10 * 60 * 1000` (10 minutes).
 - If new user: creates record with role 'affected'. If existing: updates OTP & expiry.
 - Securely prints OTP to server console (terminal-only) preventing client-side inspection vulnerabilities.
- 5 **User Enters OTP:** User enters the 6-digit code received (+ Name if first-time user).
- 6 **Verification & Token Issuance (POST /api/auth/verify-otp):**
 - Validates: User exists? Does `user.otp === otp` match? Has it expired (`otpExpires < Date.now()`)?
 - On success: Wipes `user.otp = undefined` to prevent replay attacks, sets `isVerified: true`.
 - Generates signed JSON Web Token: `jwt.sign({ id: user._id }, JWT_SECRET, { expiresIn: '7d' })`.
- 7 **Client Storage & Redirection:** JWT token and user profile are saved in `localStorage`; user is redirected to the SOS Distress Center or Affected Dashboard.

Role 2: Responders (NGO, Field Volunteer, Admin)

1 Registration (POST /api/auth/register):

- Payload: { name/orgName, email, password, role, phone, skills, vehicleType }.
- Backend validates role is within ['volunteer', 'ngo', 'admin'].
- Checks for duplicate account: `User.findOne({ email })` → rejects with 400 Email already registered if duplicate.
- Mongoose pre-save hook encrypts raw password using `bcrypt.hash(password, 10)`.

2 Login (POST /api/auth/login):

- Payload: { email, password }.
- Query retrieves user with explicitly selected password hash: `User.findOne({ email }).select('+password')`.
- Verifies hash using `await bcrypt.compare(password, user.password)`.
- If password mismatch or deactivated account → returns 401 Unauthorized.
- If valid → returns signed JWT containing User ID and role payload.
- Frontend routes the authenticated session directly to the designated dashboard (/ngo/dashboard, /volunteer/dashboard, or /admin/dashboard).

6. Core Programming Concepts: Try / Catch / Throw

Robust disaster applications must never crash when unexpected errors occur (such as database disconnection, network dropouts, or malformed user input). We implement defensive programming using JavaScript's `try...catch...throw` construct.

6.1 Theoretical Meaning

- **try { ... }:** Wraps a block of code where exceptions might happen (e.g., asynchronous database lookups, network requests, or JSON parsing).
- **throw:** Programmatically triggers an intentional error or exception when business rules or safety conditions fail.
- **catch (error) { ... }:** The rescue block that catches any thrown error, logs the stack trace, and handles the situation gracefully without terminating the server process.

6.2 Real Code Implementation in ReliefLink

File Reference: `backend/controllers/authController.js` (Lines 16–54)

```
// In backend/controllers/authController.js
const sendOTP = async (req, res) => {
  try {
    // 1. TRY BLOCK: Execute risky operational logic safely
    const { phone } = req.body;
    // Defensive check: manual error response if validation fails
    if (!phone) {
      return res.status(400).json({ success: false, message: 'Phone number is required' });
    }
    const otp = generateOTP();
    const otpExpires = new Date(Date.now() + 10 * 60 * 1000);
    // Asynchronous database lookup (can throw if MongoDB connection drops)
    let user = await User.findOne({ phone });
    if (!user) {
      user = new User({ name: 'User', phone, role: 'affected', otp, otpExpires, isVerified: false });
    } else {
      user.otp = otp;
      user.otpExpires = otpExpires;
    }
    // Database write operation (can throw if schema validation fails)
    await user.save();
    // Successful JSON response
    res.json({ success: true, message: 'OTP sent successfully. Please check your SMS.' });
  } catch (error) {
    // 2. CATCH BLOCK: If ANY error happens above, execution jumps here instantly!
    // The server does NOT crash; it logs and returns an HTTP 500 status to the client
    console.error('sendOTP Error:', error.message);
    res.status(500).json({ success: false, message: error.message });
  }
};
```

6.3 Example of throw in the Project

File Reference: `frontend/src/context/LanguageContext.jsx` (Line 52)

```
// In frontend/src/context/LanguageContext.jsx
export const useLanguage = () => {
  const context = useContext(LanguageContext);
  // If a developer tries to call this hook outside its Provider, THROW an error!
  if (!context) {
    throw new Error('useLanguage must be used within a LanguageProvider');
  }
  return context;
};
```

Viva Defense Explanation: "If a component invokes `useLanguage()` without being wrapped by `LanguageProvider`, the `throw new Error(...)` intentionally halts the rogue execution and bubbles up a clear, readable message to the error boundary, pinpointing the exact architectural issue."